

Parallel Programming HW2

108062329 何品樺

1. Implementation

a. Pthread

我主要參考講義 CH3 中 Thread pool 的實作。在 task 分配上，定義 $\text{block_size} = 1 \ll 9$, $\text{number_of_tasks} = (\text{width} * \text{height}) / \text{block_size}$ 初始化 task 參數：

```
// Initialize tasks
for (int i = 0; i < num_tasks; i++){
    task_params[i].start = i * blockSize;
    task_params[i].end = task_params[i].start + blockSize;
    if (i == num_tasks - 1){
        task_params[i].end = width * height;
    }
    //printf("e: %d %d %d\n", i, task_params[i].start, task_params[i].end);
    task_params[i].img = image;

    threadpool.queue[i].function = calcMandelbrot;
    threadpool.queue[i].argument = (void*)&task_params[i];
}
threadpool.count = num_tasks;
```

b. Hybrid

我參考講義 CH5 中 Work pool 的實作。每個 process 會被分配到 $\text{width} \ll 2$ 大小的工作，再如同 1.a 中，去切分每個 thread 的工作，這裡 $\text{thread_block_size} = 1 \ll 4$ 。

為了不讓 Master process (rank 0) 閒著，我用 pthread 另外 split 一個 thread 來負責 Work assignment 的工作，這樣 Master process (rank 0) 的 Main thread 就可以像其他 Process 接受工作，但要注意的是由於是 rank 0，無法直接用原來的 send/recv 來 communicate (雖然 self communicate 可行，但接受 task index 時會錯誤接收到來自其他 process 的 result)，所以 rank 0 的 Main thread 必須自己拿工作，並使用 mutex 來避免 race condition：

```
while(1){
    pthread_mutex_lock(&idx_lock);
    while(!start){
        pthread_cond_wait(&notify, &idx_lock);
    }
    if(p_task_idx < p_num_tasks){
        task_idx = p_task_idx;
        p_task_idx++;
    }
    else {
        bk = true;
    }
    pthread_mutex_unlock(&idx_lock);
    if(bk)break;
}
```

Worker (Main thread)

```
do {
    MPI_Recv(buf, (row_block_size << 1), MPI_INT, MPI_ANY_SOURCE, RESULT_TAG, MPI_COMM_WORLD, &status);
    int size_cpy = row_block_size;
    if(proc_to_task[status.MPI_SOURCE] == p_num_tasks - 1){
        size_cpy = width * height - proc_to_task[status.MPI_SOURCE] * row_block_size;
    }
    memcpy((void*)(f_image + proc_to_task[status.MPI_SOURCE] * row_block_size), (void*)buf, size_cpy * count--);
    pthread_mutex_lock(&idx_lock);
    if(p_task_idx < p_num_tasks){
        MPI_Send(&p_task_idx, 1, MPI_INT, status.MPI_SOURCE, DATA_TAG, MPI_COMM_WORLD);
        proc_to_task[status.MPI_SOURCE] = p_task_idx;
        count++;
        p_task_idx++;
    }
    else {
        MPI_Send(&p_task_idx, 1, MPI_INT, status.MPI_SOURCE, TERMINATE_TAG, MPI_COMM_WORLD);
    }
    pthread_mutex_unlock(&idx_lock);
} while(count > 0);
```

Master (task-assigning thread)

c. Vectorization

在 1.a 和 1.b 中我都有用 Vectorization 來加速 Mandelbrot set 的計算，使用 double 所以最多一次算兩個：

```
void CalcMandelbrot(void* args){
    task_params_t* param = (task_params_t*)args;
    int start = param->start;
    int end = param->end;
    int* image = param->img;

    int size = ((end - start) >> 1) << 1;
    int k = 0;
    int i, j;
    j = start / width;
    i = start % width;
    __m128i i_v, j_v, w_v_cmp, w_v;
    i_v = _mm_set_epi32(i + 1, i, i + 1, i);
    j_v = _mm_set1_epi32(j);
    w_v_cmp = _mm_set1_epi32(width-1);
    w_v = _mm_set1_epi32(width);

    __m128d lower_v, left_v, i_step_v, j_step_v;
    lower_v = _mm_set1_pd(lower);
    left_v = _mm_set1_pd(left);
    j_step_v = _mm_set1_pd(((upper - lower) / height));
    i_step_v = _mm_set1_pd(((right - left) / width));

    __m128i add_re, add_ite, add_cry;
    add_re = _mm_set1_epi64x(1);
    add_ite = _mm_set1_epi32(2);
    add_cry = _mm_set1_epi32(1);

    __m128d mul2 = _mm_set1_pd((double)2.0);
    __m128d cmp = _mm_set1_pd((double)4.0);
```

```
for (k = start; k < start + size; k += 2) {
    __m128d x_v, y_v, x0_v, y0_v, temp_v, temp1_v, temp2_v, temp3_v, mask_val, len_sq;
    __m128i repeats_v, mask_ite, tempi1_v, tempi2_v;

    repeats_v = _mm_set1_epi64x(0);

    __m128i carry_mask = _mm_cmpgt_epi32(i_v, w_v_cmp);
    __m128i c_j_v = _mm_add_epi32(add_cry, j_v);
    __m128i c_i_v = _mm_sub_epi32(i_v, w_v);

    i_v = _mm_blendv_epi8(i_v, c_i_v, carry_mask);
    j_v = _mm_blendv_epi8(j_v, c_j_v, carry_mask);

    y0_v = _mm_add_pd(_mm_mul_pd(_mm_cvtepi32_pd(j_v), j_step_v), lower_v);
    x0_v = _mm_add_pd(_mm_mul_pd(_mm_cvtepi32_pd(i_v), i_step_v), left_v);

    x_v = _mm_set1_pd((double)0.0);
    y_v = _mm_set1_pd((double)0.0);

    i_v = _mm_add_epi32(i_v, add_ite);

    mask_val = _mm_cmplt_pd(mul2, cmp);

    __m128d last_x_squared, last_y_squared, new_x_v, new_y_v, x_diff_v, y_diff_v;
    last_x_squared = _mm_set1_pd((double)0.0);
    last_y_squared = _mm_set1_pd((double)0.0);
```

1. 初始化 constants & i, j

2. 在每一次 loop 開始初始化 x, y，以及使用 mask(是否進位)和 blendv 計算新的 i, j

```
int r = 0;
while (r < iters) {
    mask_ite = _mm_castpd_si128(mask_val);
    if(_mm_test_all_zeros(mask_ite, mask_ite))break;

    // temp = x * x - y * y + x0
    temp3_v = _mm_sub_pd(last_x_squared, last_y_squared);
    temp_v = _mm_add_pd(temp3_v, x0_v); // temp

    // y = 2 * x * y + y0
    temp2_v = _mm_mul_pd(_mm_add_pd(x_v, x_v), y_v);
    temp3_v = _mm_add_pd(temp2_v, y0_v);

    // masked x, y replacements
    x_v = _mm_blendv_pd(x_v, temp_v, mask_val);
    y_v = _mm_blendv_pd(y_v, temp3_v, mask_val);

    // masked repeats increments
    tempi1_v = _mm_add_epi64(repeats_v, add_re);
    mask_ite = _mm_castpd_si128(mask_val);
    repeats_v = _mm_blendv_epi8(repeats_v, tempi1_v, mask_ite);
    r++;

    last_x_squared = _mm_mul_pd(x_v, x_v);
    last_y_squared = _mm_mul_pd(y_v, y_v);
    // length_squared = x * x + y * y
    len_sq = _mm_add_pd(last_x_squared, last_y_squared);
    mask_val = _mm_cmplt_pd(len_sq, cmp);
}
repeats_v = _mm_shuffle_epi32(repeats_v, _MM_SHUFFLE(3, 1, 2, 0));
_mm_storel_epi64((__m128i*)&(image[param->write + k - start]), repeats_v);
```

3. Iterate part, 使用 mask 和 blendv 來固定平方 ≥ 4 的 repeat，當 mask 裡都是 0 的時候，即兩邊平方都 ≥ 4 ，while 迴圈停止

2. Experiment & Analysis

a. Methodology

i. System Spec

I run on Apollo workstation

ii. Performance Metrics

I use testcase **strict-21** for time profiling.

For pthread version, I use timespec in time.h to calculate program time, since using clock will result in value scales up with the number of threads. For hybrid version I use MPI_Wtime() at the beginning and the end of the main function then average the time over processes to get the total time.

b. Plots: Scalability & Load Balancing

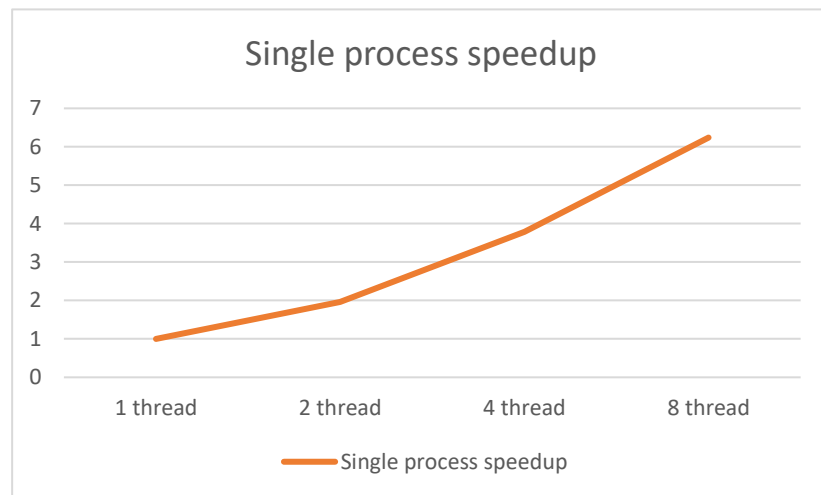


Fig 1. Single-process multi-thread

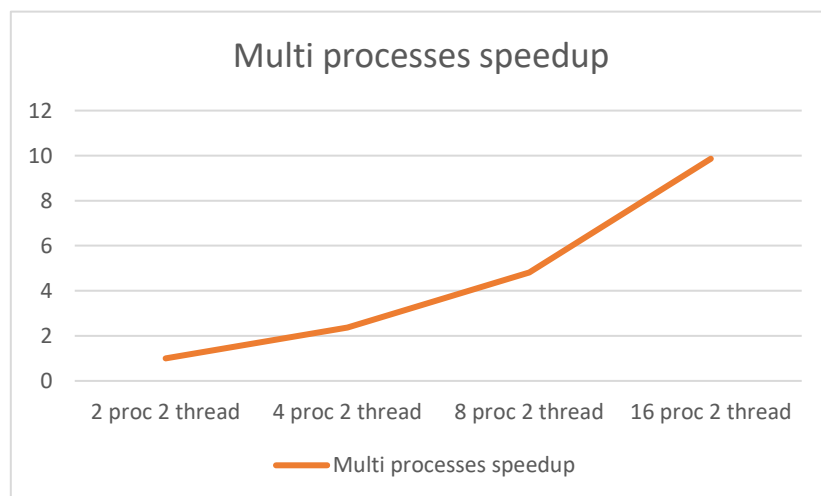


Fig 2. Multi-process multi-thread speedup

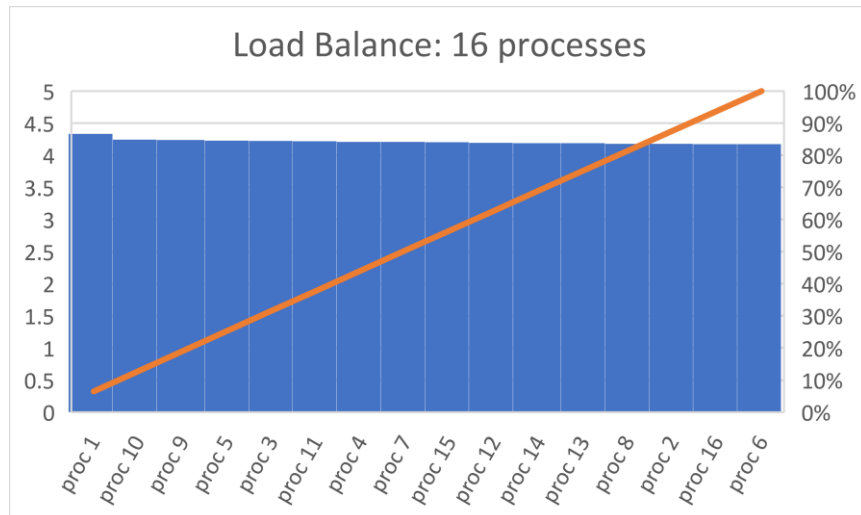


Fig 3. Load balance chart

To test load balance, I use MPI_Wtime() before and after omp section to calculate the CPU time for each process:

```
MPI_Recv(&task_idx, 1, MPI_INT, 0, MPI_ANY_TAG, MPI_COMM_WORLD, &status);
while(status.MPI_TAG == DATA_TAG){
    if(task_idx == p_num_tasks - 1){
        t_num_tasks = (width * height - task_idx * row_block_size) / thread_block_size;
    }
    s_b = MPI_Wtime();
    #pragma omp parallel for num_threads(num_threads) schedule(dynamic)
    for(int i = 0; i < t_num_tasks; i++){
        task_params[i].start = task_idx * row_block_size + i * thread_block_size;
        task_params[i].end = task_params[i].start + thread_block_size;
        task_params[i].write = i * thread_block_size;
        if(i == t_num_tasks - 1){
            if(task_idx == p_num_tasks - 1){
                task_params[i].end = width * height;
            }
            else {
                task_params[i].end = (task_idx + 1) * row_block_size;
            }
        }
        task_params[i].img = buf;
        CalcMandelbrot((void*)&task_params[i]);
    }
    int size_send = task_params[t_num_tasks-1].end - task_params[0].start;
    t_b = MPI_Wtime();
    cpu_time += t_b - s_b;

    MPI_Send(buf, size_send, MPI_INT, 0, RESULT_TAG, MPI_COMM_WORLD);
    MPI_Recv(&task_idx, 1, MPI_INT, 0, MPI_ANY_TAG, MPI_COMM_WORLD, &status);
}
```

Slave processes

```
pthread_mutex_unlock(&idx_lock);
if(bk)break;
int* task_write = f_image + task_idx * row_block_size;

s_b = MPI_Wtime();
#pragma omp parallel for num_threads(num_threads) schedule(dynamic)
for(int i = 0; i < t_num_tasks; i++){
    task_params[i].start = task_idx * row_block_size + i * thread_block_size;
    task_params[i].end = task_params[i].start + thread_block_size;
    task_params[i].write = i * thread_block_size;
    if(i == t_num_tasks - 1){
        if(task_idx == p_num_tasks - 1){
            task_params[i].end = width * height;
        }
        else {
            task_params[i].end = (task_idx + 1) * row_block_size;
        }
    }
    task_params[i].img = task_write;
    CalcMandelbrot((void*)&task_params[i]);
}
t_b = MPI_Wtime();
cpu_time += t_b - s_b;
```

Master process, main(worker) thread

c. Discussion

i. Scalability

From Fig 1. and Fig 2. we can see that the scalabilities are quite well on this implementation. In Fig 2. we can get almost 10x speedup from 2 processes to 16 processes. However, I can't get the number of processes beyond 32 when the number of threads is greater than 1, thus my plots end with only 8 threads in single-process, and 16

processes, 2 threads in multi-process (I can't even use 16 processes and 4 threads and I don't know why).

ii. Load Balance

From Fig 3. we see that the loads are balance over processes as the orange line is almost linear and the times are all close to 4.25.

d. Experience & Conclusion

這次作業花的時間主要都在 vectorization 上，我選擇自己寫 vectorize 的程式，過程除了須大量翻閱 Reference，Debug 也踩了很多坑，例如把 vector 內 element 的 index 順序搞反(與一般我們直覺 larger index 會在右邊相反)。最後雖然是完成了，但可惜計算上仍有許多可以優化的地方沒做到。對 thread programming 也熟悉許多，之前寫 pthread 程式常常還沒開始跑就 hang 在那邊，大多是沒有寫好 thread join 或是一些奇奇怪怪的原因，Debug 起來相當痛苦，希望未來不要再遇到了。